Resolución — Taller SQL sobre HR 8 — Consultas jerárquicas y WITH

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — El organigrama con sangría

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar el organigrama completo de la empresa, empezando por quien no tiene jefe, con cada persona sangrada según su nivel.

Mostrar: apellido con sangría y nivel, en orden de organigrama: cada jefe seguido de su equipo y los hermanos por apellido.

Idea. `START WITH` marca la raíz del árbol, `CONNECT BY PRIOR` dice cómo se pasa de una fila a sus hijas y `LEVEL` es la profundidad (1 en la raíz).

Pasos.

1. `START WITH e.manager_id IS NULL` arranca en quien no tiene jefe: Steven King.
2. `CONNECT BY PRIOR e.employee_id = e.manager_id` dice que las hijas de una fila son los empleados cuyo `manager_id` es el `employee_id` de esa fila.
3. `LEVEL` da la profundidad y `LPAD(' ', 2 * (LEVEL - 1))` agrega dos espacios por cada nivel.
4. `ORDER SIBLINGS BY e.last_name` ordena a los hermanos entre sí sin romper el árbol.

```
SELECT LPAD(' ', 2 * (LEVEL - 1)) || e.last_name AS organigrama,
       LEVEL AS nivel
FROM hr.employees e
START WITH e.manager_id IS NULL
CONNECT BY PRIOR e.employee_id = e.manager_id
ORDER SIBLINGS BY e.last_name;
```

Resultado en HR. 107 filas y 2 columnas; se muestran las primeras 6 filas:

```
ORGANIGRAMA  NIVEL
-----------  -----
King             1
  Cambrault      2
    Bates        3
    Bloom        3
    Fox          3
    Kumar        3
```

Ojo. Un `ORDER BY` normal rompería el organigrama: `ORDER SIBLINGS BY` solo ordena dentro de cada grupo de hermanos. El árbol de HR tiene 107 filas y 4 niveles. `LPAD(' ', 0)` devuelve `NULL`, pero concatenar con `NULL` no borra el apellido, y por eso el presidente sale sin sangría.

Ejercicio 2 — El equipo de una persona

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar todo el equipo de Neena Yang (`employee_id` 101): quienes dependen de ella directamente y quienes dependen de esas personas, a cualquier profundidad. Neena Yang no se incluye.

Mostrar: identificador, nombre completo y distancia en niveles desde Neena Yang, en orden de organigrama.

Idea. El recorrido puede empezar en cualquier fila, no solo en la raíz del organigrama: `START WITH e.employee_id = 101` toma esa fila como raíz de su propio árbol.

Pasos.

1. `START WITH e.employee_id = 101` arranca en Neena Yang.
2. `CONNECT BY PRIOR e.employee_id = e.manager_id` baja por las hijas, nivel tras nivel.
3. `LEVEL - 1` es la distancia: 1 para quien depende directamente.
4. `WHERE LEVEL > 1` deja fuera a la raíz, que es la persona de partida.

```
SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS empleado,
       LEVEL - 1 AS distancia
FROM hr.employees e
WHERE LEVEL > 1
START WITH e.employee_id = 101
CONNECT BY PRIOR e.employee_id = e.manager_id
ORDER SIBLINGS BY e.last_name;
```

Resultado en HR. 11 filas y 3 columnas; se muestran las primeras 6 filas:

```
EMPLOYEE_ID  EMPLEADO         DISTANCIA
-----------  ---------------  ---------
        204  Hermann Brown            1
        108  Nancy Gruenberg          1
        110  John Chen                2
        109  Daniel Faviet            2
        113  Luis Popp                2
        111  Ismael Sciarra           2
```

Ojo. Una condición en el `WHERE` se aplica después de armar el árbol y no recorta ramas. Quitar a Nancy Gruenberg con el `WHERE` deja a sus cinco subordinados en el resultado, mientras que ponerlo en el `CONNECT BY` los quita junto con ella: de las 12 filas del árbol quedan 11 en un caso y 6 en el otro.

Ejercicio 3 — Cuántas personas hay en cada nivel

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Contar cuántas personas hay en cada nivel del organigrama.

Mostrar: nivel y cantidad de personas, ordenados por nivel.

Idea. `LEVEL` es una pseudocolumna: se puede agrupar y contar como cualquier columna.

Pasos.

1. El recorrido es el mismo del primer ejercicio, desde quien no tiene jefe.
2. `GROUP BY LEVEL` forma un grupo por nivel.
3. `COUNT(*)` cuenta cuántas filas del árbol caen en cada nivel.

```
SELECT LEVEL AS nivel,
       COUNT(*) AS personas
FROM hr.employees e
START WITH e.manager_id IS NULL
CONNECT BY PRIOR e.employee_id = e.manager_id
GROUP BY LEVEL
ORDER BY LEVEL;
```

Resultado en HR. 4 filas y 2 columnas:

```
NIVEL  PERSONAS
-----  --------
    1         1
    2        14
    3        82
    4        10
```

Ojo. Los cuatro niveles suman las 107 personas. El nivel 3 es el más grande, con 82: incluye a los 40 empleados de Shipping que no son gerentes y a los 30 representantes de ventas, que dependen de gerentes de nivel 2.

Ejercicio 4 — La cadena de mando de IT

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Para los empleados del departamento IT (`department_id` 60), mostrar la cadena de mando completa desde el presidente hasta cada uno, con los apellidos separados por flechas.

Mostrar: empleado y cadena, en orden de organigrama.

Idea. `SYS_CONNECT_BY_PATH(col, separador)` arma el camino desde la raíz hasta la fila actual, uniendo el valor de `col` de cada fila del camino con el separador.

Pasos.

1. El recorrido parte del presidente y baja por todo el organigrama.
2. `SYS_CONNECT_BY_PATH(e.last_name, ' > ')` arma, para cada fila, la cadena de apellidos desde la raíz.
3. El `WHERE e.department_id = 60` se queda con los empleados de IT después de armar el árbol.
4. `ORDER SIBLINGS BY e.last_name` ordena a los hermanos.

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       SYS_CONNECT_BY_PATH(e.last_name, ' > ') AS cadena
FROM hr.employees e
WHERE e.department_id = 60
START WITH e.manager_id IS NULL
CONNECT BY PRIOR e.employee_id = e.manager_id
ORDER SIBLINGS BY e.last_name;
```

Resultado en HR. 5 filas y 2 columnas:

```
EMPLEADO         CADENA
---------------  ------------------------------
Alexander James   > King > Garcia > James
Valli Jackson     > King > Garcia > James > ...
Bruce Miller      > King > Garcia > James > ...
Diana Nguyen      > King > Garcia > James > ...
David Williams    > King > Garcia > James > ...
```

Ojo. El camino siempre empieza con el separador, por eso cada cadena arranca con una flecha. Y como el `WHERE` se aplica después de armar el árbol, la cadena incluye a personas que no son de IT, como Steven King y Lex Garcia.

Variante. Para quitar el separador inicial se corta el texto con SUBSTR desde la posición 4:

```
SELECT e.first_name || ' ' || e.last_name AS empleado,
       SUBSTR(SYS_CONNECT_BY_PATH(e.last_name, ' > '), 4) AS cadena
FROM hr.employees e
WHERE e.department_id = 60
START WITH e.manager_id IS NULL
CONNECT BY PRIOR e.employee_id = e.manager_id
ORDER SIBLINGS BY e.last_name;
```

Ejercicio 5 — Hacia arriba: la cadena de jefes

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar la cadena de jefes de Sundar Ande (`employee_id` 166), desde Sundar Ande hasta el presidente.

Mostrar: nivel, persona y cargo (`job_id`).

Idea. Si se invierte la condición del `CONNECT BY`, el recorrido sube: el padre de una fila es el empleado cuyo `employee_id` es el `manager_id` de esa fila.

Pasos.

1. `START WITH e.employee_id = 166` arranca en Sundar Ande.
2. `CONNECT BY PRIOR e.manager_id = e.employee_id` pasa de una fila a la de su jefe.
3. `LEVEL` cuenta los pasos hacia arriba, empezando en 1.

```
SELECT LEVEL AS nivel,
       e.first_name || ' ' || e.last_name AS persona,
       e.job_id
FROM hr.employees e
START WITH e.employee_id = 166
CONNECT BY PRIOR e.manager_id = e.employee_id;
```

Resultado en HR. 3 filas y 3 columnas:

```
NIVEL  PERSONA            JOB_ID
-----  -----------------  -------
    1  Sundar Ande        SA_REP
    2  Alberto Errazuriz  SA_MAN
    3  Steven King        AD_PRES
```

Ojo. El único cambio frente al recorrido hacia abajo es de qué lado está `PRIOR`: `PRIOR e.manager_id = e.employee_id` sube y `PRIOR e.employee_id = e.manager_id` baja. El recorrido termina solo cuando llega a una fila sin jefe.

Ejercicio 6 — El tamaño de cada rama

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Para cada persona que depende directamente del presidente, contar cuántas personas forman su rama del organigrama, contando a esa persona.

Mostrar: apellido de quien encabeza la rama y cantidad de personas, de mayor a menor cantidad y, si empatan, por apellido.

Idea. `CONNECT_BY_ROOT col` devuelve el valor de `col` en la raíz de la rama a la que pertenece la fila. Una vez calculada en una vista en línea, se agrupa por ella para contar cada rama.

Pasos.

1. `START WITH e.manager_id = 100` arranca en las 14 personas que dependen directamente del presidente.

2. `CONNECT BY PRIOR e.employee_id = e.manager_id` baja por cada rama.
3. `CONNECT_BY_ROOT e.last_name` es el apellido de quien encabeza la rama de cada fila; se calcula en una vista en línea.
4. La consulta externa agrupa por ese apellido y cuenta con `COUNT(*)`.

```
SELECT x.gerente,
       COUNT(*) AS personas
FROM (SELECT CONNECT_BY_ROOT e.last_name AS gerente
      FROM hr.employees e
      START WITH e.manager_id = 100
      CONNECT BY PRIOR e.employee_id = e.manager_id) x
GROUP BY x.gerente
ORDER BY personas DESC, x.gerente;
```

Resultado en HR. 14 filas y 2 columnas; se muestran las primeras 6 filas:

```
GERENTE   PERSONAS
--------  --------
Yang            12
Fripp            9
Kaufling         9
Mourgos          9
Vollman          9
Weiss            9
```

Ojo. Las 14 ramas suman 106 personas: toda la empresa menos el presidente, que está por encima de todas. `CONNECT_BY_ROOT` no puede escribirse directamente en el `GROUP BY` (da `ORA-00979`), por eso se calcula antes, en la vista en línea. Si el `START WITH` fuera el presidente, todas las filas tendrían la misma raíz y saldría una sola fila.

Ejercicio 7 — Quiénes no tienen subordinados

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Contar cuántos empleados no tienen subordinados (son hojas del árbol) y cuántos sí los tienen.

Mostrar: tipo y cantidad de personas, de mayor a menor cantidad.

Idea. `CONNECT_BY_ISLEAF` vale 1 en las filas que no tienen hijas en el árbol y 0 en las demás.

Pasos.

1. El recorrido es el de todo el organigrama, desde quien no tiene jefe.
2. `CONNECT_BY_ISLEAF` marca con 1 a quienes no tienen subordinados.
3. `DECODE` traduce ese 1 y ese 0 a un texto.
4. Se agrupa por ese texto y se cuenta.

```
SELECT DECODE(CONNECT_BY_ISLEAF, 1, 'Sin subordinados',
                                 'Con subordinados') AS tipo,
       COUNT(*) AS personas
FROM hr.employees e
START WITH e.manager_id IS NULL
CONNECT BY PRIOR e.employee_id = e.manager_id
GROUP BY DECODE(CONNECT_BY_ISLEAF, 1, 'Sin subordinados',
                'Con subordinados')
ORDER BY personas DESC;
```

Resultado en HR. 2 filas y 2 columnas:

```
TIPO              PERSONAS
----------------  --------
Sin subordinados        89
Con subordinados        18
```

Ojo. Las 18 personas con subordinados son las mismas del ejercicio 4 del taller 3. Aquí no hizo falta `EXISTS`: el recorrido ya sabe si una fila tiene hijas.

Ejercicio 8 — Personas a cargo y nómina de cada equipo

Nivel: Reto · **Tablas:** `employees`

Se pide. Para cada jefe, mostrar cuántas personas tiene a su cargo, directa o indirectamente, y la nómina de su equipo contando al propio jefe.

Mostrar: jefe, personas a cargo y nómina del equipo, de mayor a menor cantidad de personas a cargo y, si empatan, por apellido.

Idea. Sin `START WITH`, `CONNECT BY` toma cada fila como raíz de su propio árbol. Con `CONNECT_BY_ROOT`, cada fila del recorrido sabe de qué jefe cuelga y se puede sumar por jefe.

Pasos.

1. La vista en línea `t` recorre el organigrama desde cada empleado, porque no tiene `START WITH`.
2. `CONNECT_BY_ROOT e.employee_id` da la raíz de cada recorrido, y `e.salary` el sueldo de cada persona alcanzada.
3. Se une `t` con `hr.employees m` por esa raíz y se agrupa por jefe.
4. `COUNT(*) - 1` son las personas a cargo (sin contar al jefe) y `SUM(t.salary)` es la nómina del equipo con el jefe.
5. `HAVING COUNT(*) > 1` deja solo a quienes tienen a alguien a cargo.

```
SELECT m.first_name || ' ' || m.last_name AS jefe,
       COUNT(*) - 1 AS a_cargo,
       SUM(t.salary) AS nomina_equipo
FROM hr.employees m
JOIN (SELECT CONNECT_BY_ROOT e.employee_id AS raiz,
             e.salary
      FROM hr.employees e
      CONNECT BY PRIOR e.employee_id = e.manager_id) t
  ON t.raiz = m.employee_id
GROUP BY m.employee_id, m.first_name, m.last_name
HAVING COUNT(*) > 1
ORDER BY a_cargo DESC, m.last_name;
```

Resultado en HR. 18 filas y 3 columnas; se muestran las primeras 6 filas:

```
JEFE            A_CARGO  NOMINA_EQUIPO
--------------  -------  -------------
Steven King         106         691416
Neena Yang           11         109816
Adam Fripp            8          33600
Payam Kaufling        8          31500
Kevin Mourgos         8          28800
Shanta Vollman        8          32400
```

Ojo. El presidente tiene a cargo a las otras 106 personas y la nómina de su equipo es la de toda la empresa: 691416, la misma suma del ejercicio 2 del taller 2. Salen 18 jefes, los mismos que en el ejercicio anterior.

Ejercicio 9 — El organigrama de ventas por salario

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar el organigrama del área de ventas: cada gerente de ventas (`SA_MAN`) seguido de su equipo, con los gerentes y los representantes ordenados de mayor a menor salario.

Mostrar: nombre completo con sangría y salario, en orden de organigrama.

Idea. `START WITH` puede arrancar en varias filas a la vez: aquí, en cada gerente de ventas. Cada una es la raíz de su propia rama.

Pasos.

1. `START WITH e.job_id = 'SA_MAN'` arranca en los cinco gerentes de ventas.
2. `CONNECT BY PRIOR e.employee_id = e.manager_id` baja a sus representantes.
3. `LPAD(' ', 3 * (LEVEL - 1))` sangra tres espacios por nivel.
4. `ORDER SIBLINGS BY e.salary DESC, e.last_name` ordena a los hermanos por salario, de mayor a menor.

```
SELECT LPAD(' ', 3 * (LEVEL - 1)) || e.first_name || ' ' || e.last_name
         AS organigrama,
       e.salary
FROM hr.employees e
START WITH e.job_id = 'SA_MAN'
CONNECT BY PRIOR e.employee_id = e.manager_id
ORDER SIBLINGS BY e.salary DESC, e.last_name;
```

Resultado en HR. 35 filas y 2 columnas; se muestran las primeras 6 filas:

```
ORGANIGRAMA           SALARY
--------------------  ------
John Singh             14000
   Sean Tucker         10000
   David Bernstein      9500
   Peter Hall           9000
   Christopher Olsen    8000
   Nanette Cambrault    7500
```

Ojo. Los cinco gerentes son hermanos de primer nivel y se ordenan entre sí por salario, y cada uno arrastra a sus representantes, también ordenados por salario. Kimberely Grant aparece en la rama de Eleni Zlotkey aunque no tiene departamento: el organigrama sigue a `manager_id`, no al departamento.

Ejercicio 10 — El mismo organigrama con WITH recursivo

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Obtener el organigrama del primer ejercicio con una expresión `WITH` recursiva, que es la forma estándar de SQL, en lugar de `CONNECT BY`.

Mostrar: apellido con sangría y nivel, en el mismo orden del primer ejercicio.

Idea. Una expresión `WITH` recursiva tiene dos partes unidas con `UNION ALL`: la parte ancla, que da las filas de partida, y la parte recursiva, que se une con el propio resultado para dar el nivel siguiente.

Pasos.

1. La parte ancla toma a quien no tiene jefe, con nivel 1.
2. La parte recursiva une `hr.employees` con `arbol` por `manager_id` y suma 1 al nivel.
3. `SEARCH DEPTH FIRST BY last_name SET orden` numera las filas en orden de organigrama, con los hermanos por apellido.
4. La consulta final sangra con `LPAD` y ordena por esa numeración.

```
WITH arbol (employee_id, last_name, nivel) AS (
  SELECT e.employee_id, e.last_name, 1
  FROM hr.employees e
  WHERE e.manager_id IS NULL
  UNION ALL
  SELECT h.employee_id, h.last_name, a.nivel + 1
  FROM hr.employees h
  JOIN arbol a ON h.manager_id = a.employee_id
)
SEARCH DEPTH FIRST BY last_name SET orden
SELECT LPAD(' ', 2 * (a.nivel - 1)) || a.last_name AS organigrama,
       a.nivel
FROM arbol a
ORDER BY a.orden;
```

Resultado en HR. 107 filas y 2 columnas; se muestran las primeras 6 filas:

```
ORGANIGRAMA  NIVEL
-----------  -----
King             1
  Cambrault      2
    Bates        3
    Bloom        3
    Fox          3
    Kumar        3
```

Ojo. Da las mismas 107 filas del primer ejercicio. `WITH` recursivo es estándar y funciona en otros motores; `CONNECT BY` es propio de Oracle, más corto y trae pseudocolumnas como `LEVEL` y `SYS_CONNECT_BY_PATH`. Sin la cláusula `SEARCH` el orden de las filas no está garantizado.

Ejercicio 11 — Una expresión WITH para no repetir la consulta

Nivel: Intermedio · **Tablas:** `employees`, `departments`

Se pide. Listar los departamentos cuya nómina total supera el promedio de las nóminas de todos los departamentos que tienen empleados.

Mostrar: departamento, nómina y promedio de las nóminas (dos decimales), de mayor a menor nómina.

Idea. `WITH nombre AS (consulta)` le da nombre a una consulta para usarla varias veces en la principal, como una tabla temporal que solo vive durante esa sentencia.

Pasos.

1. `nomina` calcula la nómina total de cada departamento, una sola vez.
2. La consulta principal lee `nomina` para listar los departamentos.
3. Una subconsulta lee `nomina` otra vez para calcular el promedio de las nóminas.
4. El filtro `n.total > promedio` y el orden por nómina descendente completan la consulta.

```
WITH nomina AS (SELECT d.department_name,
                       SUM(e.salary) AS total
                FROM hr.employees e
                JOIN hr.departments d ON d.department_id = e.department_id
                GROUP BY d.department_name)
SELECT n.department_name,
       n.total,
       ROUND((SELECT AVG(x.total) FROM nomina x), 2) AS promedio_nominas
FROM nomina n
WHERE n.total > (SELECT AVG(x.total) FROM nomina x)
ORDER BY n.total DESC;
```

Resultado en HR. 2 filas y 3 columnas:

```
DEPARTMENT_NAME   TOTAL  PROMEDIO_NOMINAS
---------------  ------  ----------------
Sales            304500          62219.64
Shipping         156400          62219.64
```

Ojo. Sin `WITH` habría que repetir la consulta agrupada en el `FROM` y otra vez dentro de la subconsulta. Ojo con el significado: este es el promedio de once nóminas de departamento, no el promedio de salarios de la empresa (6461.83).

Ejercicio 12 — Reto: los tres equipos más grandes, con empates

Nivel: Reto · **Tablas:** `employees`

Se pide. Entre las personas que dependen directamente del presidente, mostrar los tres equipos más grandes (contando a quien lo encabeza) y la cantidad de niveles de cada uno. Si hay un empate en el tercer lugar, deben salir todos los empatados.

Mostrar: apellido de quien encabeza el equipo, personas y niveles, de mayor a menor cantidad de personas y, si empatan, por apellido.

Idea. Se combinan tres piezas: `CONNECT_BY_ROOT` para saber a qué rama pertenece cada fila, `GROUP BY` para contar por rama y `FETCH FIRST ... WITH TIES` para no cortar un empate.

Pasos.

1. La vista en línea `x` recorre cada rama desde `START WITH e.manager_id = 100` y trae, para cada fila, el apellido de la raíz y su nivel.
2. La vista `g` agrupa por ese apellido y calcula `COUNT(*)` (personas) y `MAX(x.nivel)` (niveles).
3. `ORDER BY personas DESC FETCH FIRST 3 ROWS WITH TIES` se queda con los tres primeros y con todos los que empatan con el tercero.
4. El `ORDER BY` de la consulta externa fija el orden en que se muestran, con los empatados por apellido.

```
SELECT g.gerente,
       g.personas,
       g.niveles
FROM (SELECT x.gerente,
             COUNT(*) AS personas,
             MAX(x.nivel) AS niveles
      FROM (SELECT CONNECT_BY_ROOT e.last_name AS gerente,
                   LEVEL AS nivel
            FROM hr.employees e
            START WITH e.manager_id = 100
            CONNECT BY PRIOR e.employee_id = e.manager_id) x
      GROUP BY x.gerente
      ORDER BY personas DESC
      FETCH FIRST 3 ROWS WITH TIES) g
ORDER BY g.personas DESC, g.gerente;
```

Resultado en HR. 6 filas y 3 columnas:

```
GERENTE   PERSONAS  NIVELES
--------  --------  -------
Yang            12        3
Fripp            9        2
Kaufling         9        2
Mourgos          9        2
Vollman          9        2
Weiss            9        2
```

Ojo. El tercer lugar está empatado: cinco ramas tienen 9 personas, así que salen seis filas y no tres. Con `ROWS ONLY` el corte entre esas cinco sería arbitrario. El primer equipo, el de Neena Yang, es también el más profundo: tres niveles frente a dos de los demás.